

Introduction to Computer Programming

Fundamentals of C for Complete Beginners

Amit Kumar Singh
Assistant Professor (ECE)
School of Naval Architecture and Ocean Engineering
Indian Maritime University Visakhapatnam Campus, Sabbavaram Mandal
Visakhapatnam, Andhra Pradesh – 531 035
amitkumarsingh@imu.ac.in

August 17, 2026

Outline

- 1 Introduction to Computer Organization
- 2 Evolution of Operating Systems
- 3 Machine, Assembly & High-Level Languages
- 4 Software & Hardware Trends; Programming Methodologies
- 5 Program Development in C
- 6 Structured Programming: Algorithms & Pseudo-code
- 7 Anatomy of a C Program
- 8 The C Standard Library
- 9 Data Types in C
- 10 Operators in C
- 11 Input and Output in C
- 12 Control Structures
- 13 Writing Your First C Programs

What is a Computer?

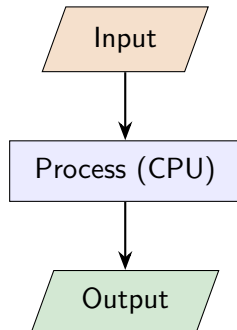
Definition

A **computer** is an electronic machine that accepts **data** (input), processes it according to a set of **instructions** (a program), and produces useful **information** (output).

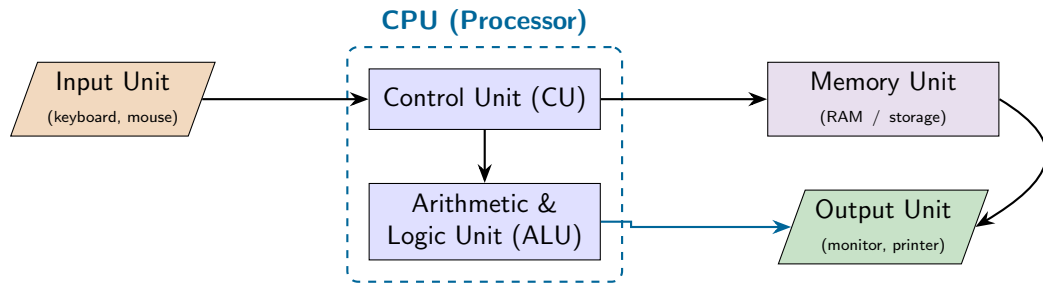
- It follows the simple cycle:

INPUT → PROCESS → OUTPUT

- It cannot “think” — it only does exactly what the program tells it, very fast and without mistakes.
- Programming* is the art of writing those instructions.



Computer Organization: The Building Blocks



- **CPU** = “brain”. **CU** directs traffic; **ALU** does the maths & comparisons.
- **Memory** temporarily holds data & instructions the CPU is working on.

Hardware vs. Software & Types of Memory

Hardware

The physical parts you can *touch* — CPU, RAM, hard disk, keyboard, monitor.

Software

The *programs* (instructions) that tell the hardware what to do — e.g. Windows, a game, a C program.

Primary Memory (fast, temporary)

- **RAM** — working memory, lost when power is off (volatile).
- **ROM** — read-only, permanent start-up instructions.

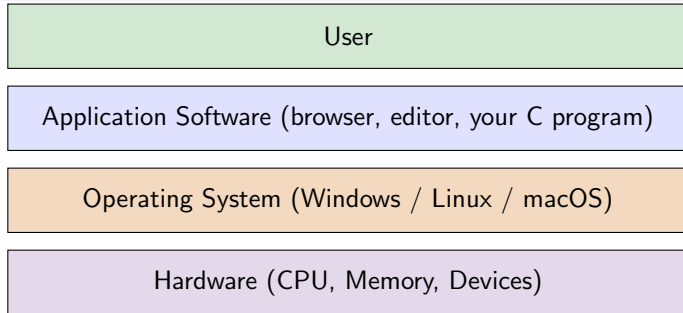
Secondary Memory (slow, permanent)

- Hard disk, SSD, USB drive — stores files even without power.

What is an Operating System (OS)?

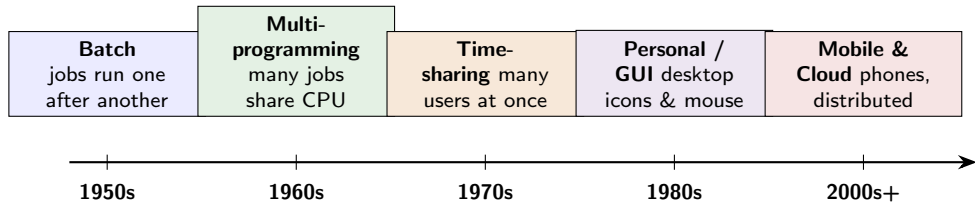
Operating System

An OS is the **master software** that manages the hardware and gives other programs a place to run. It is the bridge between **you**, the **programs**, and the **hardware**.



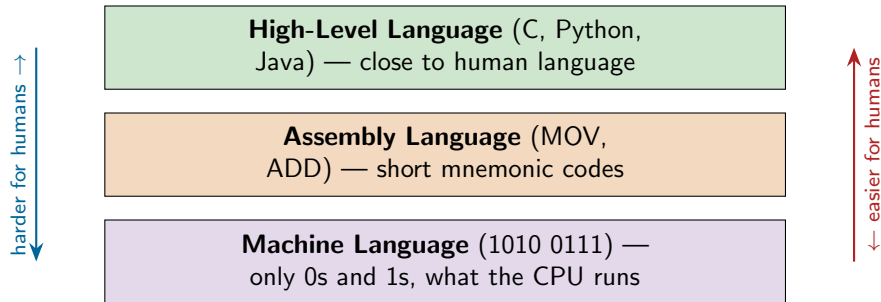
Examples of tasks: managing memory, running programs, handling files, controlling devices.

Evolution of Operating Systems: A Timeline



- Each stage made the computer **easier to use** and able to do **more at once**.
- Modern OSes let many programs and users run *simultaneously* on the same machine.

Levels of Programming Languages



- The CPU understands **only** machine language (binary).
- We write in a high-level language; a **translator** converts it to machine code.

Comparing the Three Levels

Feature	Human friendliness	Example (add 2 numbers)
Machine	Very hard (all 0/1)	10110000 01100001
Assembly	Hard (mnemonics)	MOV AL, 61h ADD AL, BL
High-level	Easy (English-like)	c = a + b;

Key idea

High-level languages like **C** let us write *one* readable line instead of many cryptic machine instructions — and the same C program can run on many different machines.

Key Software & Hardware Trends

Hardware trends

- **Moore's Law:** transistor count roughly doubles $\sim$ every 2 years $\rightarrow$ faster, cheaper chips.
- Multi-core CPUs, GPUs.
- Smaller & mobile: phones, embedded, IoT.
- Cloud data-centres provide computing “on tap”.

Software trends

- Open-source software & reusable libraries.
- Mobile apps and web applications.
- Artificial Intelligence & data science.
- Higher-level, safer languages built *on top of C*.

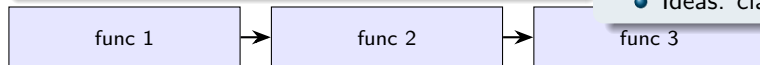
Hardware gets faster and cheaper; software gets smarter and more connected — and **C** sits underneath much of it.

Procedural vs. Object-Oriented Programming

Procedural (e.g. C)

Program = a sequence of **procedures** / **functions** that operate on data.

- Think: a *recipe* — do step 1, then 2, then 3.
- Data and functions are separate.



Object-Oriented (e.g. C++, Java)

Program = a set of **objects** that bundle data *and* the functions acting on it.

- Think: real-world *objects* (a Car that can `start()`, `stop()`).
- Ideas: class, object, inheritance.

Object: Car
data: speed
methods:
`start()`, `stop()`

In this course we focus on the **procedural** style using **C**.

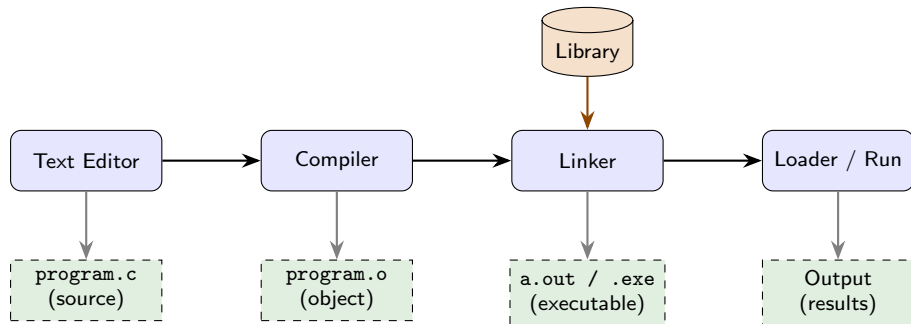
Why C?

- Created by **Dennis Ritchie** at Bell Labs (early 1970s).
- **Powerful** yet **small** — close to the hardware but readable.
- The “mother language”: Linux, databases, and parts of Windows are written in C.
- Learning C teaches you how a computer *really* works.

C is a great first language because

- It is **fast**.
- It is used **everywhere**.
- Its ideas carry over to C++, Java, Python, ...

From Source Code to Running Program



One command does it all (using GCC)

```
gcc program.c -o program
then ./program
```

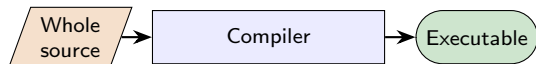
Each step's output feeds the next. If the compiler reports **errors**, fix them and compile again — this “edit → compile → run” loop is normal!

Translators: Compiler vs. Interpreter

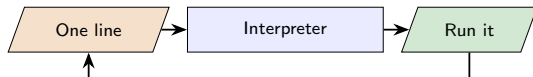
Why we need a translator

A **translator** converts our high-level C into machine code. The two main kinds are the **compiler** and the **interpreter**.

Compiler (C uses this)



Interpreter (e.g. Python)

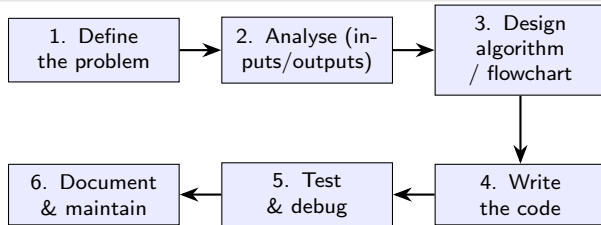


Compiler	Interpreter
Translates the <i>whole</i> program at once	Translates <i>line by line</i>
Produces a separate executable file	No separate executable file
Faster; all errors reported together	Slower; errors reported line-wise

Steps in Problem Solving

Before writing code, plan the solution

Good programmers think first, then type. Solving a problem on a computer follows a clear set of steps.



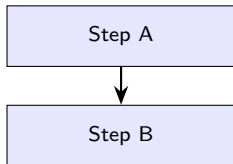
- **Analysis** asks: what are the *inputs*, the *outputs*, and the *constraints*?
- Only step 4 is actual coding — most of the thinking happens *before* it.

Structured Programming

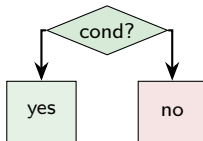
Idea

Build every program from just **three** basic control structures. This keeps code clear, correct, and easy to read (no “spaghetti” jumps).

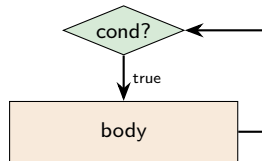
1. Sequence (do in order)



2. Selection (choose a path)



3. Repetition (loop)



Algorithm and Pseudo-code

Algorithm

A finite, step-by-step procedure to solve a problem — *before* writing any code.

Algorithm: add two numbers

- 1 Start
- 2 Read two numbers a and b
- 3 Compute $\text{sum} = a + b$
- 4 Display sum
- 5 Stop

Pseudo-code

An informal, English-like sketch of the code — language independent.

```
BEGIN
    INPUT a, b
    sum  $\leftarrow$  a + b
    OUTPUT sum
END
```

Good pseudo-code translates almost line-by-line into C.

Characteristics of a Good Algorithm

- **Input** — zero or more well-defined inputs.
- **Output** — at least one useful output.
- **Definiteness** — every step is precise and unambiguous.
- **Finiteness** — it must end after a finite number of steps.
- **Effectiveness** — each step is basic enough to be carried out.

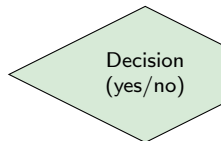
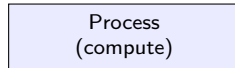
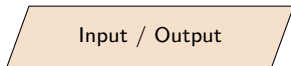
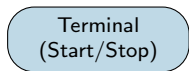
Rule of thumb

If a stranger can follow your steps and get the *same* correct answer every time, it is a good algorithm.

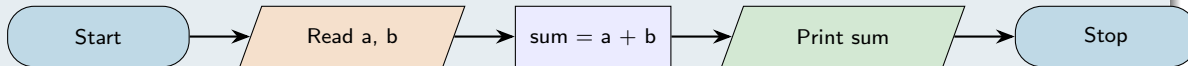
Why it matters

A clear algorithm makes the C code almost write itself — and makes bugs easy to spot.

Flowchart Symbols You Should Know



Flowchart: add two numbers



The Skeleton of Every C Program

```
#include <stdio.h>    /* 1: header */

int main(void)        /* 2: start  */
{                    /* 3: body   */
    /* your statements go here */

    return 0;        /* 4: exit   */
}
```

- ➊ **Preprocessor directive** — `#include` pulls in a library (here `stdio.h` for input/output).
- ➋ `main()` — execution always begins here; every program has exactly one.
- ➌ **Body** — the statements between `{ }`; each ends with a semicolon `;`
- ➍ `return 0;` — tells the OS the program finished successfully.

Learn this shape first — every program we write just fills in the body.

The C Character Set and Tokens

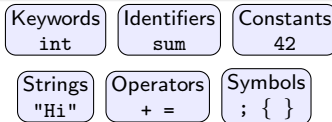
Character set

The symbols C understands:

- Letters A--Z, a--z
- Digits 0--9
- Special symbols + - * / = % ; , ...
- Whitespace (space, tab, newline)

Tokens — the smallest units

Characters group into **tokens**:



A compiler first breaks your code into these tokens.

Keywords: C's Reserved Words

What is a keyword?

A **keyword** has a fixed, special meaning in C. There are **32** of them — you cannot use them as your own names. C is **case-sensitive**: `int` is a keyword, `INT` is not.

<code>auto</code>	<code>break</code>	<code>case</code>	<code>char</code>	<code>const</code>	<code>continue</code>	<code>default</code>	<code>do</code>
<code>double</code>	<code>else</code>	<code>enum</code>	<code>extern</code>	<code>float</code>	<code>for</code>	<code>goto</code>	<code>if</code>
<code>int</code>	<code>long</code>	<code>register</code>	<code>return</code>	<code>short</code>	<code>signed</code>	<code>sizeof</code>	<code>static</code>
<code>struct</code>	<code>switch</code>	<code>typedef</code>	<code>union</code>	<code>unsigned</code>	<code>void</code>	<code>volatile</code>	<code>while</code>

The ones we use most as beginners: `int`, `float`, `char`, `if`, `else`, `for`, `while`, `return`.

Identifiers: Naming Your Variables

Identifier

The **name** you give to a variable, function, or constant.

Rules

- Use letters, digits, and underscore `_`.
- Must *start* with a letter or `_` (not a digit).
- No spaces and no special symbols.
- Cannot be a keyword.
- Case-sensitive: `age` $\neq$ `Age`.

Valid

`total`

`marks1`

`_count`

`avgScore`

Invalid

`2sum` (starts with digit)

`my age` (has a space)

`float` (keyword)

`rate%` (bad symbol)

Tip: choose *meaningful* names like `total`, not `t`.

The C Standard Library

What is it?

A ready-made collection of **functions** that come with C, so you don't reinvent the wheel. You get access to them with `#include <header.h>`.

Header	Purpose	Example functions
stdio.h	input / output	printf, scanf, getchar
stdlib.h	general utilities	malloc, rand, abs, atoi
math.h	mathematics	sqrt, pow, sin, ceil
string.h	text handling	strlen, strcpy, strcmp
ctype.h	character tests	isdigit, toupper

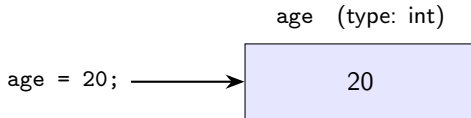
e.g. `#include <math.h>` then `y =`

`sqrt(25.0);` gives `y = 5.0`.

Variables and Data Types

Variable

A **named box** in memory that stores a value. You must declare its *type* first: `int age = 20;`



- The **type** decides what values fit and how much memory is used.
- Choose the smallest type that comfortably holds your data.

The Basic Data Types

Type	Stores	Typical size	Example
int	whole numbers	4 bytes	<code>int n = 42;</code>
float	real numbers (decimals)	4 bytes	<code>float g = 9.8f;</code>
double	real numbers (more precise)	8 bytes	<code>double pi = 3.14159;</code>
char	a single character	1 byte	<code>char grade = 'A';</code>

Modifiers

short, long, unsigned adjust range/size, e.g.
unsigned int (no negatives), long int (bigger).

A character is a small number

'A' is stored as the integer 65 (its ASCII code).

Size and Range of Each Variable

Every type occupies a fixed amount of memory

The **size** (in bytes) decides the **range** of values a variable can hold. Sizes are typical for a modern computer.

Type	Size	Bits	Approx. range	Specifier
char	1 byte	8	−128 to 127	%c
int	4 bytes	32	-2.1×10^9 to 2.1×10^9	%d
float	4 bytes	32	~6–7 significant digits	%f
double	8 bytes	64	~15 significant digits	%lf

Find the size yourself

The sizeof operator returns a type's size in bytes:

```
1 printf("%d", sizeof(int));  
2 // prints 4
```

Constants: Values That Never Change

Constant

A value that is fixed and **cannot be changed** while the program runs (e.g. π , the number of days in a week). Using constants makes code safer and easier to read.

Using the `const` keyword:

```
const float PI = 3.14159;  
const int DAYS = 7;
```

Using `#define` (a macro):

```
1 #define MAX 100  
2 #define PI 3.14159
```

- `const` makes a *typed* read-only variable; `#define` does a plain text substitution before compiling.
- By convention, constants are written in UPPERCASE. Trying to change one is an error.

Variable Scope: Local vs. Global

```
int x = 5;           /* global */

int main(void) {
    int y = 10;      /* local */
    printf("%d %d", x, y);
    return 0;
}
```

Scope = where a variable is visible

- **Local** — declared *inside* a function; usable only there.
- **Global** — declared *outside* all functions; usable everywhere.

Prefer local variables — they keep parts of the program independent and reduce accidental bugs.

Arithmetic Operators

Operator	Meaning	Example	Result
+	addition	$7 + 3$	10
-	subtraction	$7 - 3$	4
*	multiplication	$7 * 3$	21
/	division	$7 / 3$	2 (integer!)
%	modulus (remainder)	$7 \% 3$	1

Beginner trap: integer division

$7 / 3$ gives 2, *not* 2.33, because both are integers. Use $7.0 / 3$ to get 2.333... The % operator works only on integers.

Operator Precedence (Order of Operations)

- C follows maths-like precedence:
 - $\ast$ / $\%$ **before** $+$ $-$
 - Same level $\rightarrow$ left to right.
- Use **parentheses** to be explicit and clear.

Example

```
2 + 3 * 4  $\rightarrow$  2 + 12  $\rightarrow$  14  
(2 + 3) * 4  $\rightarrow$  20
```

Assignment & shortcuts

- $x = 5$; stores 5 in x.
- $x += 2$; means $x = x + 2$;
- $x++$; increases x by 1.
- $x--$; decreases x by 1.

Relational and Logical Operators

Relational — compare two values

Result is **true (1)** or **false (0)**.

Op	Meaning	Example
==	equal to	5 == 5 → 1
!=	not equal	5 != 3 → 1
>	greater than	5 > 3 → 1
<	less than	5 < 3 → 0
>=	greater/equal	5 >= 5 → 1
<=	less/equal	4 <= 3 → 0

Logical — combine conditions

Used to join or invert true/false tests.

Op	Meaning	True when
&&	AND	both are true
	OR	at least one is true
!	NOT	flips true/false

>= 18 && age < 60) is true only for ages 18 to 59.

Assignment and Increment Operators

Operator	Meaning
<code>x = 5</code>	assign 5 to x
<code>x += 2</code>	<code>x = x + 2</code>
<code>x -= 3</code>	<code>x = x - 3</code>
<code>x *= 4</code>	<code>x = x * 4</code>
<code>x /= 5</code>	<code>x = x / 5</code>

```
1 int x = 10;  
2 x += 5;    // 15  
3 x -= 3;    // 12  
4 x *= 2;    // 24  
5 x /= 4;    // 6
```

Final value of x is 6.

A Peek at Bitwise Operators

Working at the level of bits

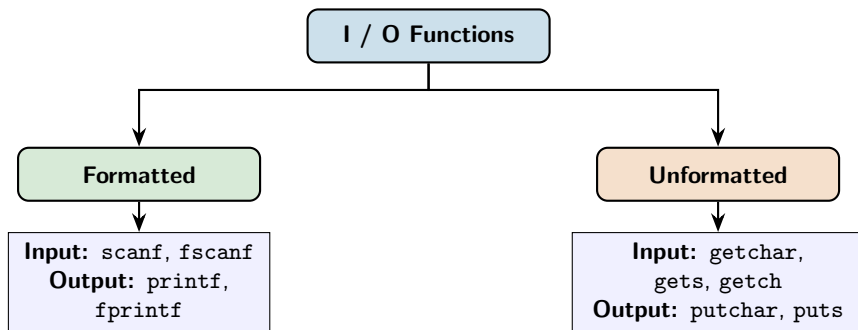
Bitwise operators act on the individual **0/1 bits** of an integer. They are common in *embedded systems* and *hardware control* — handy to recognise even as a beginner.

```
int a = 5;    // 0101
int b = 3;    // 0011

a & b;        // 0001 = 1  (AND)
a | b;        // 0111 = 7  (OR)
a ^ b;        // 0110 = 6  (XOR)
~a;           // flips all bits
a << 1;       // 1010 = 10 (x2)
a >> 1;       // 0010 = 2  (/2)
```

- & AND — 1 only if *both* bits are 1.
- | OR — 1 if *any* bit is 1.
- ^ XOR — 1 if the bits *differ*.
- ~ NOT — flips every bit.
- << left shift — multiplies by 2.
- >> right shift — divides by 2.

Classification of I/O Functions



- **Formatted** functions use *format specifiers* (`%d`, `%f`, ...) to control layout — our focus.
- **Unformatted** functions read/write raw characters or strings without formatting.

Output with printf — Format Specifiers

Specifier	Used for	Example
%d	integer	<code>printf("%d", 42);</code> → 42
%f	float / double	<code>printf("%f", 3.14);</code> → 3.140000
%c	single character	<code>printf("%c", 'A');</code> → A
%s	string (text)	<code>printf("%s", "Hi");</code> → Hi

Width & precision

`%5d` — at least 5 columns wide.

`%.2f` — 2 digits after the point.

`printf("%.2f", 3.14159);` → 3.14

Escape sequences

`\n` new line `\t` tab

`\\` backslash `\"` quote

Input with scanf

```
1 int age;
2 printf("Enter your age: ");
3 scanf("%d", &age);           /* note the & (address of age) */
4 printf("Next year you will be %d\n", age + 1);
```

- scanf reads typed input and stores it in a variable.
- **Crucial:** put an & (“address of”) before the variable — &age.
- The specifier must match the type: %d int, %f float, %c char, %s string.

Reading several values

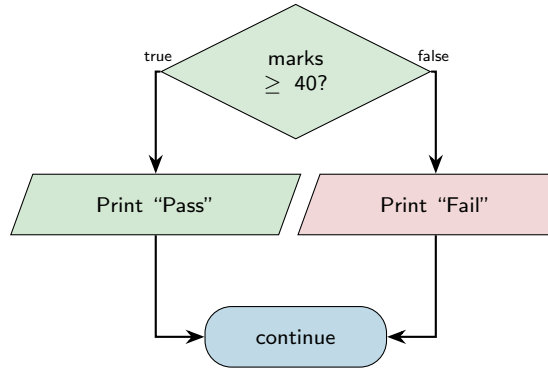
`scanf("%d %f", &n, &x);` reads an integer then a float.

Sample run: →

Making Decisions: if / if-else

```
if (marks >= 40) {  
    printf("Pass\n");  
} else {  
    printf("Fail\n");  
}
```

- The condition in () is either **true** or **false**.
- Comparisons: == != < > <= >=
- **Trap:** == tests equality; = *assigns*!



Choosing Among Many: switch

```
switch (choice) {  
    case 1:  
        printf("Add\n");  
        break;  
    case 2:  
        printf("Subtract\n");  
        break;  
    default:  
        printf("Invalid\n");  
}
```

- Compares one variable against several case values.
- `break`; stops “falling through” to the next case — **don't forget it**.
- `default`: runs if nothing else matches.
- Cleaner than a long `if--else if` chain.

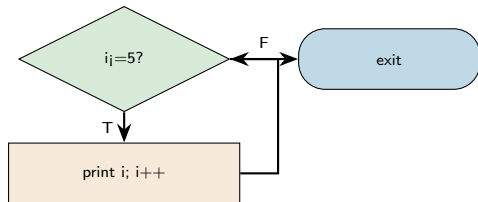
Note

case labels must be constant integers or characters (e.g. 1, 'A').

Loops: while and do--while

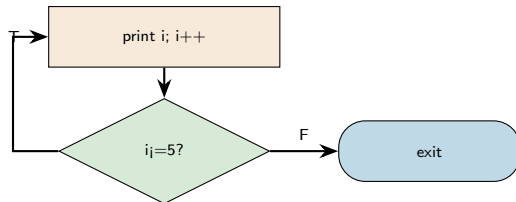
while — test *first*, may run 0 times:

```
int i = 1;
while (i <= 5) {
    printf("%d ", i);
    i++;
}
```



do--while — test *after*, runs ≥ 1 time:

```
1 int i = 1;
2 do {
3     printf("%d ", i);
4     i++;
5 } while (i <= 5);
```



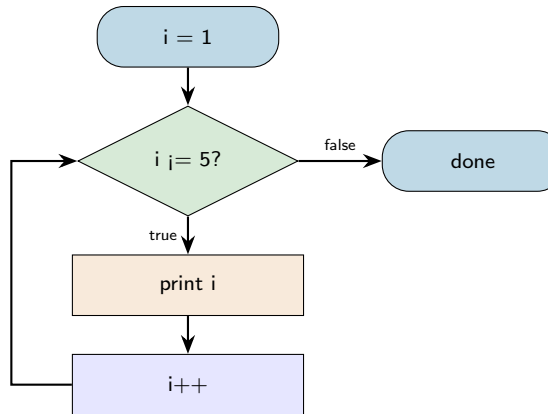
Both print 1 2 3 4 5. Use do--while when the body must run at least once.

Loops: the for Loop

```
for (i = 1; i <= 5; i++) {  
    printf("%d ", i);  
}
```

The three parts of a for loop:

- 1 **Init:** `i = 1` — runs once at the start.
- 2 **Condition:** `i <= 5` — checked before each pass.
- 3 **Update:** `i++` — runs after each pass.



A for loop is a compact while loop — best when you know the count in advance.

break and continue

break — exit the loop immediately.

```
for (i = 1; i <= 10; i++) {  
    if (i == 4) break;  
    printf("%d ", i);  
}  
// prints: 1 2 3
```

continue — skip to the next iteration.

```
1 for (i = 1; i <= 5; i++) {  
2     if (i == 3) continue;  
3     printf("%d ", i);  
4 }  
5 // prints: 1 2 4 5
```

Remember

break **leaves** the loop; continue **jumps to the next round**. Both also appear inside while/do--while; break additionally ends a switch case.

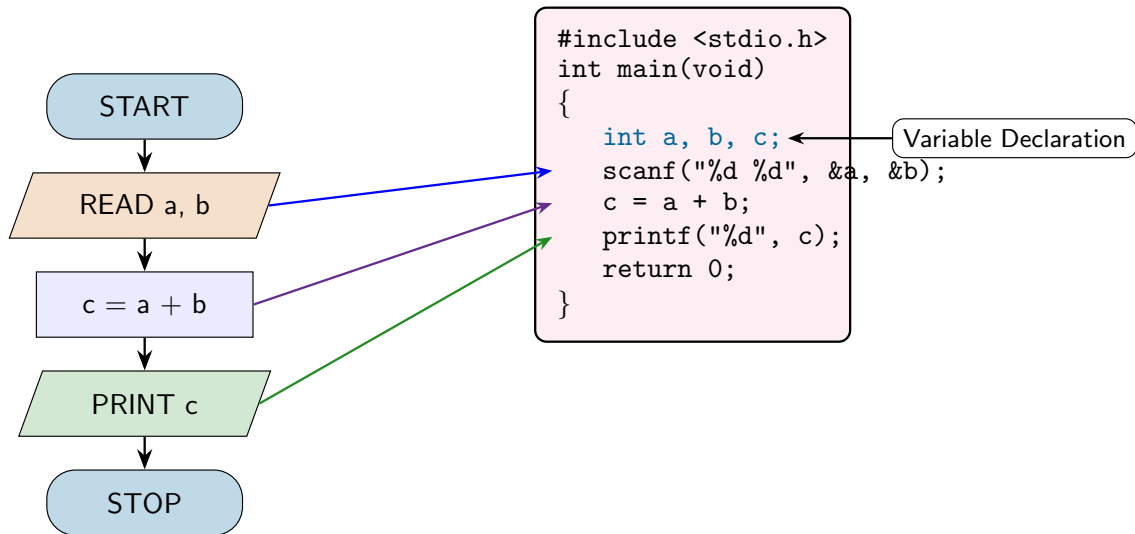
Your First C Program: “Hello, World!”

```
1 #include <stdio.h>           /* bring in input/output library */
2
3 int main(void)               /* execution starts here */
4 {
5     printf("Hello, World!\n"); /* print a message */
6     return 0;                 /* 0 = success */
7 }
```

- We now know the skeleton, keywords, data types and I/O — so we can write real programs.
- `printf` prints text; `\n` starts a new line.
- Every statement ends with a semicolon ;
- `return 0;` reports success to the OS.

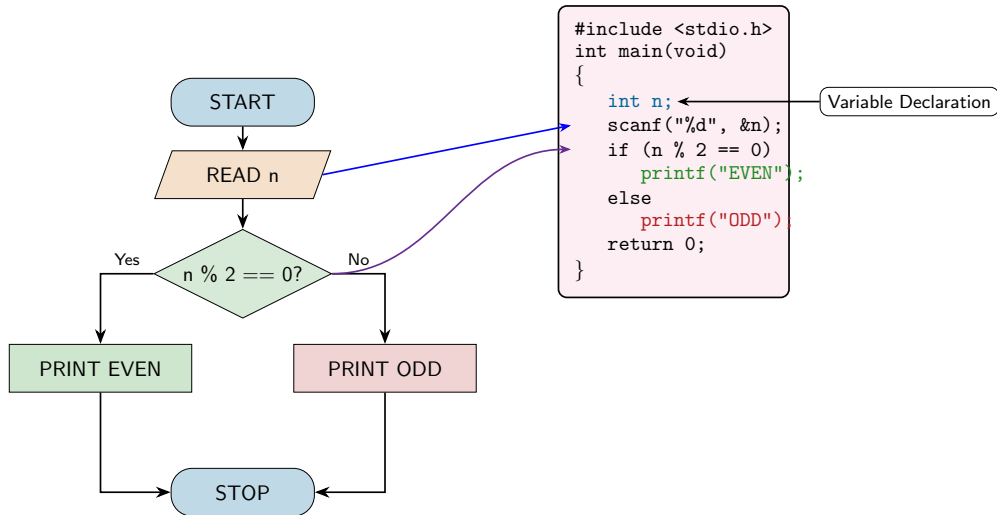
Output: Hello, World!

Worked Example 1: Adding Two Numbers



Each flowchart step on the left maps to a coloured line of C code on the right.

Worked Example 2: Even or Odd (if--else)



The green and red branches match the two printf lines. A number is even when $n \% 2$ is 0.

Worked Example 3 — Sum of 1 to N (a for loop)

```
#include <stdio.h>

int main(void)
{
    int n, i, sum = 0;

    printf("Enter N: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
        sum += i;    /* sum = sum + i */

    printf("Sum 1..%d = %d\n", n, sum);
    return 0;
}
```

Trace for N = 5

i	sum
1	1
2	3
3	6
4	10
5	15

Output:

Sum 1..5 = 15

Worked Example 4 — Simple Calculator (switch)

```
1 #include <stdio.h>
2 int main(void) {
3     float a, b;  char op;
4     printf("Enter: a op b (e.g. 6 * 7): ");
5     scanf("%f %c %f", &a, &op, &b);
6
7     switch (op) {
8         case '+': printf("= %.2f\n", a + b); break;
9         case '-': printf("= %.2f\n", a - b); break;
10        case '*': printf("= %.2f\n", a * b); break;
11        case '/':
12            if (b != 0) printf("= %.2f\n", a / b);
13            else       printf("Cannot divide by zero!\n");
14            break;
15        default: printf("Unknown operator\n");
16    }
17    return 0;
18 }
```

Summary & What Comes Next

You now know

- How a computer is organized and what an OS does.
- Machine / assembly / high-level languages.
- How a C program is written, compiled, and run.
- Data types, operators, and control structures.
- Formatted input/output with `printf/scanf`.

Practice makes perfect

- Type and run every example yourself.
- Deliberately make errors — read the messages.
- Modify examples: change values, add features.

Coming up

Functions, arrays, strings, and pointers.

Thank You

Questions?